

나의 영토 Spring Boot 프로젝트 셋업 가이드

Oct 2, 2026 · @https://m.kinolights.com/ranking/netflix

목표와 전제

이 가이드대로 만들면 `./gradlew :app-api:bootRun` 한 줄로 `http://localhost:8080/health` 가 뜨는 Java 멀티모듈 뼈대가 나온다. 도메인 코드는 아직 없고, 모듈 경계·설정·테스트 틀만 잡는다. 7번 섹션의 요청문을 Spring Boot CLI(또는 Claude Code 같은 코딩 CLI)에 그대로 붙여 넣으면 된다.

항목	값
언어	Java 21 (Kotlin 사용 안 함)
빌드	Gradle 8.x, Groovy DSL(<code>build.gradle</code>), Wrapper 포함
프레임워크	Spring Boot 3.5.x, Spring Web, Spring Data JPA, Validation, Actuator
설정	<code>application.yml</code> + 프로파일 <code>local</code> / <code>prod</code> (properties 파일 없음)
로컬 DB	H2 인메모리(<code>local</code>), 실제 DB 연결은 2단계에서 MySQL로 교체
그룹/패키지	<code>com.kobi.territory</code>
포트	8080
모듈	<code>common</code> , <code>catalog</code> , <code>exploration</code> , <code>progression</code> , <code>wardrobe</code> , <code>social</code> , <code>sharing</code> , <code>app-api</code> (도메인 분석 문서의 바운디드 컨텍스트와 1:1)

도메인 분석·애그리거트 설계는 「나의 영토 도메인 분석·애그리거트 설계」 문서를 따른다. 이 가이드는 그 문서의 "Spring Boot 멀티모듈 매핑" 섹션을 실제로 띄울 수 있는 수준으로 구체화한 것이다.

프로젝트 구조

루트는 빌드 설정만 갖고, 실행 가능한 모듈은 `app-api` 하나다. 나머지 모듈은 일반 Java 라이브러리로, `bootJar` 를 끄고 `jar` 만 만든다.

```

territory/
├─ settings.gradle
├─ build.gradle           # 공통 플러그인·의존성 버전
├─ gradle.properties
├─ gradlew, gradlew.bat, gradle/wrapper/
├─ common/                # 이벤트 인터페이스, 공통 값 객체, 에러 타입
│   └─ src/main/java/com/kobi/territory/common/
├─ catalog/               # Region Catalog (지역·아이템 정의 참조 데이터)
├─ exploration/           # Territory, ExpeditionMap, 체크인
├─ progression/           # ExplorerProgress, Collection, QuestBoard
├─ wardrobe/              # Inventory, Scene
├─ social/                # Friendship, 피드, 랭킹
├─ sharing/               # ShareCard
└─ app-api/               # 실행 모듈: Application, yaml, 전역 예외 처리,
ArchUnit
├─ src/main/java/com/kobi/territory/TerritoryApplication.java
├─ src/main/resources/application.yml
├─ src/main/resources/application-local.yml
└─ src/test/java/com/kobi/territory/ArchitectureTest.java

```

도메인 모듈 안은 네 패키지로 고정한다. 빈 패키지도 `package-info.java` 를 두어 구조가 처음부터 보이게 한다.

패키지	담는 것	의존 허용
<code>..<module>.api< code=""></module>.api<></code>	Controller, Request/Response DTO	application
<code>.. <module>.application</code>	UseCase 서비스, 트랜잭션 경계, 이벤트 발행	domain
<code>..<module>.domain< code=""></module>.domain<></code>	애그리거트, 값 객체, 도메인 이벤트, Repository 인터페이스	common만
<code>..<module>.infra< code=""></module>.infra<></code>	JPA 엔티티/구현체, 외부 클라이언트	domain, application

모듈 간 의존은 한 방향이다: 모든 도메인 모듈 → `common`, `app-api` → 모든 모듈. 도메인 모듈끼리는 직접 참조하지 않고 `common` 의 이벤트로만 통신한다(예: `exploration`이 `RegionVisited` 를 발행하고 `progression`이 구독). 예외는 `catalog` 로, 지역 코드 검증이 필요한 `exploration·wardrobe`가 읽기 전용으로 참조할 수 있다.

Gradle 설정

버전과 공통 의존성은 루트에서 한 번만 정하고, 각 모듈은 자기 의존성만 적는다.

settings.gradle

```

rootProject.name = 'territory'

include 'common'
include 'catalog'
include 'exploration'
include 'progression'
include 'wardrobe'
include 'social'
include 'sharing'
include 'app-api'

```

gradle.properties

```

org.gradle.jvmargs=-Xmx2g
org.gradle.parallel=true
org.gradle.caching=true

```

루트 build.gradle

```

plugins {
    id 'java'
    id 'org.springframework.boot' version '3.5.6' apply false
    id 'io.spring.dependency-management' version '1.1.7' apply false
}

allprojects {
    group = 'com.kobi'
    version = '0.0.1-SNAPSHOT'
    repositories { mavenCentral() }
}

subprojects {
    apply plugin: 'java'
    apply plugin: 'java-library'
    apply plugin: 'io.spring.dependency-management'

    java {
        toolchain { languageVersion = JavaLanguageVersion.of(21) }
    }

    dependencyManagement {
        imports {
            mavenBom
            org.springframework.boot.gradle.plugin.SpringBootPlugin.BOM_COORDINATES

```

```

    }
}

dependencies {
    compileOnly 'org.projectlombok:lombok'
    annotationProcessor 'org.projectlombok:lombok'
    testImplementation 'org.springframework.boot:spring-boot-starter-
test'
}

tasks.named('test') { useJUnitPlatform() }
}

```

`org.springframework.boot` 플러그인은 루트에 `apply false` 로 올려두고 `app-api` 에서만 적용한다. 그래야 도메인 모듈들이 BOM 버전은 공유하면서 `bootJar` 태스크는 생기지 않는다.

도메인 모듈 build.gradle (exploration 예시, 나머지 동일 패턴)

```

dependencies {
    api project(':common')
    implementation project(':catalog')    // exploration, wardrobe만

    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
    implementation 'org.springframework.boot:spring-boot-starter-validation'
}

```

`common/build.gradle` 은 `spring-context` 만 갖는다(이벤트 퍼블리셔 인터페이스용). `starter-web-jpa` 는 넣지 않는다.

app-api/build.gradle

```

plugins {
    id 'org.springframework.boot'
}

dependencies {
    implementation project(':common')
    implementation project(':catalog')
    implementation project(':exploration')
    implementation project(':progression')
    implementation project(':wardrobe')
    implementation project(':social')
    implementation project(':sharing')

    implementation 'org.springframework.boot:spring-boot-starter-web'
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
}

```

```

implementation 'org.springframework.boot:spring-boot-starter-validation'
implementation 'org.springframework.boot:spring-boot-starter-actuator'

runtimeOnly 'com.h2database:h2'
runtimeOnly 'com.mysql:mysql-connector-j'

testImplementation 'com.tngtech.archunit:archunit-junit5:1.3.0'
}

bootJar { archiveFileName = 'territory.jar' }

```

application.yml

기본 프로파일은 `local` 이라 아무 옵션 없이 실행하면 H2 인메모리로 뜨고, DB 접속 정보는 환경변수로만 받는다. 게임 규칙 값(하루 상한, 온보딩 예외 시간, 탈퇴 유예일)도 코드에 박지 않고 `territory.*` 아래 둘 예정이라 자리를 미리 만든다.

app-api/src/main/resources/application.yml

```

spring:
  application:
    name: territory
  profiles:
    default: local
  jpa:
    open-in-view: false
    properties:
      hibernate:
        default_batch_fetch_size: 100
  jackson:
    time-zone: Asia/Seoul

server:
  port: 8080
  shutdown: graceful

management:
  endpoints:
    web:
      exposure:
        include: health, info

territory:
  check-in:
    daily-cap: 5

```

```

onboarding-grace-hours: 72
map:
  leave-grace-days: 7
share-card:
  cache-ttl-minutes: 10

```

app-api/src/main/resources/application-local.yml

```

spring:
  datasource:
    url: jdbc:h2:mem:territory;MODE=MySQL;DB_CLOSE_DELAY=-1
    driver-class-name: org.h2.Driver
    username: sa
    password:
  jpa:
    hibernate:
      ddl-auto: create-drop
    show-sql: true
  h2:
    console:
      enabled: true
      path: /h2-console

logging:
  level:
    com.kobi.territory: DEBUG

```

app-api/src/main/resources/application-prod.yml (지금은 자리만)

```

spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true

```

territory.* 값은 @ConfigurationProperties(prefix = "territory") 레코드 하나 (TerritoryProperties)로 묶어 app-api 에 둔다. MODE=MySQL 은 나중에 MySQL로 바꿀 때 쿼리 차이를 줄이려는 것이고, Flyway는 prod에서만 켜고 마이그레이션 파일은 2단계부터 넣는다.

최소 코드

빠대 단계에서 들어가는 코드는 다섯 파일이다. 도메인 모듈에는 `package-info.java` 만 두고 애그리거트는 아직 만들지 않는다.

app-api — TerritoryApplication.java

```
package com.kobi.territory;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import
org.springframework.boot.context.properties.ConfigurationPropertiesScan;

@SpringBootApplication
@ConfigurationPropertiesScan
public class TerritoryApplication {
    public static void main(String[] args) {
        SpringApplication.run(TerritoryApplication.class, args);
    }
}
```

패키지가 `com.kobi.territory` 라 하위 모듈의 `com.kobi.territory.exploration` 같은 빈이 전부 스캔된다. 모듈마다 `@Configuration` 을 따로 만들 필요가 없다.

app-api — TerritoryProperties.java

```
package com.kobi.territory.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "territory")
public record TerritoryProperties(CheckIn checkIn, Map map, ShareCard
shareCard) {
    public record CheckIn(int dailyCap, int onboardingGraceHours) {}
    public record Map(int leaveGraceDays) {}
    public record ShareCard(int cacheTtlMinutes) {}
}
```

app-api — HealthController.java (actuator와 별개로, 설정값이 제대로 바인딩됐는지 눈으로 확인하는 용도)

```
package com.kobi.territory.api;

import com.kobi.territory.config.TerritoryProperties;
import java.util.Map;
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HealthController {
    private final TerritoryProperties props;

    public HealthController(TerritoryProperties props) { this.props = props;
}

    @GetMapping("/health")
    public Map<String, Object> health() {
        return Map.of(
            "status", "UP",
            "service", "territory",
            "dailyCap", props.checkIn().dailyCap(),
            "leaveGraceDays", props.map().leaveGraceDays()
        );
    }
}
```

common — DomainEvent.java (모듈 간 통신의 유일한 통로, 지금은 인터페이스만)

```
package com.kobi.territory.common.event;

import java.time.Instant;

public interface DomainEvent {
    Instant occurredAt();
}
```

app-api/src/test — ArchitectureTest.java

```
package com.kobi.territory;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.noClasses;

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;

@AnalyzeClasses(packages = "com.kobi.territory", importOptions =
    ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
```

```

static final ArchRule domain_depends_only_on_common =
    noClasses().that().resideInAPackage("..domain..")
        .should().dependOnClassesThat()
            .resideInAnyPackage("..api..", "..application..", "..infra..",
                "org.springframework.web..",
                "jakarta.persistence..");

@ArchTest
static final ArchRule modules_talk_only_through_common =

noClasses().that().resideInAPackage("com.kobi.territory.exploration..")
    .should().dependOnClassesThat()
        .resideInAnyPackage("com.kobi.territory.progression..",
            "com.kobi.territory.wardrobe..",
            "com.kobi.territory.social..",
            "com.kobi.territory.sharing..");
}

```

두 번째 규칙은 exploration 하나만 예시로 적었고, 도메인 모듈 여섯 전부에 같은 형태로 복제한다 (catalog는 읽기 참조가 허용되므로 금지 목록에서 뺀다). domain 패키지에서 jakarta.persistence 를 막은 것은 JPA 엔티티를 infra 에 두고 도메인 객체와 분리하겠다는 선언이다. 처음부터 분리가 부담이면 그 한 줄만 빼고 시작해도 된다.

실행과 검증

아래 여섯 개가 통과하면 뼈대는 끝이다.

```

gradle wrapper --gradle-version 8.14          # wrapper가 없을 때 한 번
./gradlew clean build                          # 컴파일 + ArchUnit 테스트
./gradlew :app-api:bootRun                     # local 프로파일, H2
./gradlew :app-api:bootRun --args='--spring.profiles.active=prod' # 환경변수
없으면 실패해야 정상

```

- ☐ ./gradlew projects 출력에 모듈 8개가 보인다
- ☐ ./gradlew clean build 가 성공하고 app-api/build/libs/territory.jar 만 생긴다(다른 모듈은 일반 jar)
- ☐ curl localhost:8080/health → {"status":"UP","dailyCap":5,"leaveGraceDays":7,...}
- ☐ curl localhost:8080/actuator/health → {"status":"UP"}
- ☐ localhost:8080/h2-console 접속, JDBC URL jdbc:h2:mem:territory 로 로그인
- ☐ ArchitectureTest 통과, 일부러 exploration에서 progression 클래스를 import하면 실패한다

자주 걸리는 것: `TerritoryProperties` 가 바인딩 안 되면 `@ConfigurationPropertiesScan` 이 빠졌거나 yml 키가 kebab-case(`daily-cap`)가 아닌 경우다. `bootJar` 태스크가 도메인 모듈에도 생기면 루트에서 `apply false` 를 빼먹은 것이다.

CLI 요청문

아래 블록을 통째로 복사해 CLI에 붙이면 된다. 위 섹션의 코드를 그대로 쓰라고 지시하므로 이 문서 링크나 내용을 함께 전달한다.

Spring Boot 멀티모듈 프로젝트 뼈대를 만들어줘. 도메인 코드는 넣지 말고 로컬호스트로 뜨는 것까지만.

제약

- Java 21, Kotlin 금지. Gradle Groovy DSL(build.gradle), Wrapper 포함.
- Spring Boot 3.5.6, 설정은 application.yml만(properties 파일 금지). 프로파일 local(기본, H2 인메모리)과 prod(환경변수 주입).
- 루트 프로젝트 territory, 그룹 com.kobi, 기본 패키지 com.kobi.territory, 포트 8080.
- 모듈 8개: common, catalog, exploration, progression, wardrobe, social, sharing, app-api.
 - app-api만 org.springframework.boot 플러그인 적용(bootJar 이름 territory.jar).
- 나머지는 java-library, 루트에서 BOM만 공유.
 - 의존: 모든 도메인 모듈 -> common. exploration, wardrobe -> catalog. app-api -> 전부. 도메인 모듈끼리 그 외 참조 금지.
 - 각 도메인 모듈은 api / application / domain / infra 네 패키지, 빈 패키지에도 package-info.java.
 - 의존성: starter-web, data-jpa, validation, actuator, lombok, h2(runtime), mysql-connector-j(runtime), archunit-junit5 1.3.0(test).

파일과 내용은 아래 문서의 "Gradle 설정", "application.yml", "최소 코드" 섹션을 그대로 사용해: settings.gradle, gradle.properties, 루트/모듈 build.gradle, application.yml, application-local.yml, application-prod.yml, TerritoryApplication, TerritoryProperties(@ConfigurationProperties 레코드, prefix territory), HealthController(GET /health, 설정값 포함 응답), common의 DomainEvent 인터페이스, app-api 테스트의 ArchitectureTest(domain은 common만 의존, 도메인 모듈 간 직접 참조 금지 - 모듈 6개 모두 규칙 작성).

끝나면 ./gradlew clean build 와 ./gradlew :app-api:bootRun 을 실행해서 curl localhost:8080/health 응답에 dailyCap 5, leaveGraceDays 7이 나오는 것까지 확인하고 결과를 보여줘.

뼈대가 올라온 다음 순서는 도메인 분석 문서의 "진행 순서" 1단계: catalog에 Region 참조 데이터 적재, exploration에 Territory 애그리거트와 체크인 API.